INFORME FASE 2

Entorno de prueba: PostgreSQL 16 en Docker – Base de Datos temporal: bodegas_atuel_paso7 (aunque también la usamos para el paso 8)

1. Objetivo y Alcance

Comparar dos sesiones PostgreSQL que compiten por existencias: Una lectura ordinaria sin bloqueo preventivo y otra mediante SELECT ... FOR UPDATE. Se presentan una simulación auxiliar que exhibe el riesgo de decisiones sobre lecturas antiguas, una prueba sobre la última botella en la tabla real stock_sala y una validación del procedimiento sp_procesar_transferencia.

El aislamiento utilizado fue READ COMMITTED. Las operaciones directas sobre stock_sala simulan el descuento de una venta; no se generaron comprobantes comerciales

2. Lectura sin bloqueo preventivo:

Se creó la tabla independiente demo_sin_bloqueo.stock con 10 botellas y 0 salidas autorizadas. Ambas sesiones abrieron transacciones y leyeron 10 mediante SELECT ordinario. Una lógica deliberadamente insegura basó las salidas en ese valor anterior y no volvió a verificar las existencias

```
PS C:\Users\Lasag\OneDrive\Documentos\AaaTINO\Universidad\3° Año\Base de Datos\Proyecto Integrador\trabaj
o_integrador_bd> docker exec -it pg-bodega-paso7 psql -U postgres -d bodegas_atuel_paso7
psql (16.15 (Debian 16.15-1.pgdg13+2))
Type "help" for help.

bodegas_atuel_paso7=# CREATE SCHEMA demo_sin_bloqueo;
CREATE SCHEMA
bodegas_atuel_paso7=# CREATE TABLE demo_sin_bloqueo.stock (
    id INTEGER PRIMARY KEY,
    cantidad INTEGER NOT NULL,
    salidas_autorizadas INTEGER NOT NULL
);
CREATE TABLE
bodegas_atuel_paso7=# INSERT INTO demo_sin_bloqueo.stock
VALUES (1, 10, 0);
INSERT 0 1
```

Creación de la tabla auxiliar y carga del stock inicial (10 botellas)

```
PS C:\Users\Lasag\OneDrive\Documentos\AaaTINO\Universidad\3° Año\Base de Datos\Proyecto Integrador\trabaj
o_integrador_bd> docker exec -it pg-bodega-paso7 psql -U postgres -d bodegas_atuel_paso7
psql (16.15 (Debian 16.15-1.pgdg13+2))
Type "help" for help.

bodegas_atuel_paso7=# BEGIN;
BEGIN
bodegas_atuel_paso7=*# SELECT cantidad
FROM demo_sin_bloqueo.stock
WHERE id = 1;
 cantidad
----------
       10
(1 row)
```

Lectura inicial de 10 botellas desde una sesión sin bloqueo preventivo

```
bodegas_atuel_paso7=*# UPDATE demo_sin_bloqueo.stock
SET cantidad = 3,
    salidas_autorizadas = salidas_autorizadas + 7
WHERE id = 1;
UPDATE 1
bodegas_atuel_paso7=*# COMMIT;
COMMIT
```

Actualización y confirmación de una de las transacciones

```
bodegas_atuel_paso7=# INSERT INTO demo_sin_bloqueo.stock
VALUES (1, 10, 0);
INSERT 0 1
bodegas_atuel_paso7=# SELECT
    cantidad AS stock_registrado,
    salidas_autorizadas,
    10 - salidas_autorizadas AS stock_real_calculado
FROM demo_sin_bloqueo.stock
WHERE id = 1;
 stock_registrado | salidas_autorizadas | stock_real_calculado
------------------+---------------------+----------------------
                3 |                  14 |                   -4
(1 row)
```

Resultado observado: 3 botellas registradas y 14 salidas autorizadas; salida lógica de -4

2.1. Ultima botella en stock_sala, sin FOR UPDATE

En una nueva base de pruebas se preparó el vino 1 en la sala 1 con cantidad = 1. A y B hicieron SELECT sin FOR UPDATE y ambas observaron 1. Luego A descontó la unidad con una operación condicionada a cantidad >= 1; B intentó el mismo UPDATE y esperó a que A confirmara. Al continuar, la condición se volvió a evaluar y B obtuvo UPDATE 0

3. Ultima botella sin FOR UPDATE

```
bodegas_atuel_ultima_botella=# BEGIN;

SHOW transaction_isolation;

SELECT cantidad
FROM stock_sala
WHERE id_sala = 1 AND cod_vino = 1;
BEGIN
 transaction_isolation
-----------------------
 read committed
(1 row)

 cantidad
----------
        1
(1 row)
```

Sesión A: Aislamiento READ COMMITED y lectura de 1 unidad

```
PS C:\Users\Lasag\OneDrive\Documentos\AaaTINO\Universidad\3° Año\Base de Datos\Proyecto Integrador\trabajo_integrador_bd> docker exec -it pg-bodeg
a-paso7 psql -U postgres -d bodegas_atuel_ultima_botella
psql (16.15 (Debian 16.15-1.pgdg13+2))
Type "help" for help.

bodegas_atuel_ultima_botella=# BEGIN;

SELECT cantidad
FROM stock_sala
WHERE id_sala = 1 AND cod_vino = 1;
BEGIN
 cantidad
----------
        1
(1 row)
```

Sesión B: También observa 1 unidad antes de la modificación

```
bodegas_atuel_ultima_botella=*# UPDATE stock_sala
SET cantidad = cantidad - 1
WHERE id_sala = 1
  AND cod_vino = 1
  AND cantidad >= 1
RETURNING cantidad;
 cantidad
----------
        0
(1 row)

UPDATE 1
```

Sesion A: UPDATE 1, stock descontando a 0

```
bodegas_atuel_ultima_botella=*# UPDATE stock_sala
SET cantidad = cantidad - 1
WHERE id_sala = 1
  AND cod_vino = 1
  AND cantidad >= 1
RETURNING cantidad;
 cantidad
----------
(0 rows)

UPDATE 0
```

Sesion B: UPDATE 0 tras confirmarse el descuento de A

En este experimento no hubo doble descuento. Aunque ambas sesiones leyeron 1, el UPDATE adquiere bloqueo de fila y la condición sobre las existencias impidió descontar una segunda unidad. Por eso debe distinguirse este resultado de la simulación auxiliar insegura de la sección 2

4. SELECT … FOR UPDATE

En la simulación auxiliar original se restablecieron 10 botellas. A ejecutó SELECT ... FOR UPDATE y mantuvo la transacción abierta; B intentó bloquear la misma fila y quedó esperando. Se observó la espera con pg_stat_activity y pg_blocking_pids(). Al hacer COMMIT A, B continuó y leyó 10, porque A solo había bloqueado la fila sin descontar unidades

```
bodegas_atuel_paso7=# BEGIN;
BEGIN
bodegas_atuel_paso7=*# SELECT cantidad
FROM demo_sin_bloqueo.stock
WHERE id = 1
FOR UPDATE;
```

La sesión A lee con FOR UPDATE y retiene el bloqueo

```
bodegas_atuel_paso7=# BEGIN;
BEGIN
bodegas_atuel_paso7=*# SELECT cantidad
FROM demo_sin_bloqueo.stock
WHERE id = 1
FOR UPDATE;
```

La sesión B intenta adquirir el mismo bloqueo

5. Ultima botella con FOR UPDATE

Se restituyó cantidad = 1 en stock_sala. A inició una transacción y realizó SELECT ... FOR UPDATE, obteniendo 1 y reteniendo el bloqueo. B ejecutó la misma consulta bloqueante y quedó esperando antes de poder tomar una decisión sobre la venta

```
bodegas_atuel_ultima_botella=# UPDATE stock_sala
SET cantidad = 1
WHERE id_sala = 1
  AND cod_vino = 1;

SELECT id_sala, cod_vino, cantidad
FROM stock_sala
WHERE id_sala = 1
  AND cod_vino = 1;
UPDATE 1
 id_sala | cod_vino | cantidad
---------+----------+----------
       1 |        1 |        1
(1 row)

bodegas_atuel_ultima_botella=# BEGIN;

SHOW transaction_isolation;

SELECT cantidad
```

Se restablece una unidad para repetir el escenario

```
SELECT cantidad
FROM stock_sala
WHERE id_sala = 1
  AND cod_vino = 1
FOR UPDATE;
BEGIN
 transaction_isolation
-----------------------
 read committed
(1 row)

 cantidad
----------
        1
(1 row)
```

Sesion A READ COMMITED y SELECT … FOR UPDATE devuelve 1

Durante la espera de B, A realizó un UPDATE condicionado, descontó la única unidad y confirmó. Solo entonces B obtuvo el bloqueo y pudo leer el valor actualizado.

```
bodegas_atuel_ultima_botella=*# COMMIT;

SELECT id_sala, cod_vino, cantidad
FROM stock_sala
WHERE id_sala = 1 AND cod_vino = 1;
COMMIT
 id_sala | cod_vino | cantidad
---------+----------+----------
       1 |        1 |        0
(1 row)

bodegas_atuel_ultima_botella=# BEGIN;

SELECT cantidad
FROM stock_sala
WHERE id_sala = 1
  AND cod_vino = 1
FOR UPDATE;
BEGIN
 cantidad
----------
        0
(1 row)
```

Sesion B luego de la confirmación de A, FOR UPDATE devuelve 0

```
SELECT CASE
    WHEN cantidad >= 1 THEN 'VENTA PERMITIDA'
    ELSE 'VENTA RECHAZADA: STOCK INSUFICIENTE'
END AS resultado
FROM stock_sala
WHERE id_sala = 1
  AND cod_vino = 1;

COMMIT;

-- Verificación final
SELECT id_sala, cod_vino, cantidad
FROM stock_sala
WHERE id_sala = 1
  AND cod_vino = 1;
             resultado
-------------------------------------
 VENTA RECHAZADA: STOCK INSUFICIENTE
(1 row)

COMMIT
 id_sala | cod_vino | cantidad
---------+----------+----------
       1 |        1 |        0
(1 row)
```

Decision de B venta rechazada por stock insuficiente, saldo final 0

La segunda sesión no intentó descontar la botella después de observar 0: la decisión de no vender se tomó con el valor actualizado. El experimento evidencia lectura bloqueante y coordinación entre transacciones

6. Aplicación en el procedimiento del proyecto

En una prueba independiente con el procedimiento real sp_procesar_transferencia, las transferencias T1=2 y T2=3 intentaron mover 7 botellas cada una desde un depósito con 10. Se observó que B esperaba a A. Cuando A confirmó, B fue rechazada por RN-11 (stock insuficiente: disponible 3, solicitado 7). La verificación final confirmó T1 procesado, T2 terminado, depósito 3 y sala 7. Esta evidencia complementa la simulación auxiliar.

```
PS C:\Users\Lasag\OneDrive\Documentos\AaaTINO\Universidad\3° Año\Base de Datos\Proyecto Integrador\trabaj
o_integrador_bd> docker exec -it pg-bodega-paso7 psql -U postgres -d bodegas_atuel_paso7
psql (16.15 (Debian 16.15-1.pgdg13+2))
Type "help" for help.

bodegas_atuel_paso7=# SELECT
    pid,
    state,
    wait_event_type,
    wait_event,
    pg_blocking_pids(pid) AS bloqueado_por
FROM pg_stat_activity
WHERE datname = 'bodegas_atuel_paso7'
ORDER BY pid;
 pid |        state        | wait_event_type |  wait_event   | bloqueado_por
-----+---------------------+-----------------+---------------+---------------
 242 | idle                | Client          | ClientRead    | {}
 251 | idle                | Client          | ClientRead    | {}
 259 | idle                | Client          | ClientRead    | {}
 334 | idle                | Client          | ClientRead    | {}
 347 | idle in transaction | Client          | ClientRead    | {}
 355 | active              | Lock            | transactionid | {347}
 382 | active              |                 |               | {}
(7 rows)
```

Monitoreo de una sesión en espera mediante pg_stat_activity y pg_blocking_pids()

6.1. Nivel de aislamiento utilizado

Se utilizó READ COMMITTED, el aislamiento predeterminado de PostgreSQL. Las consultas ordinarias ven datos confirmados al comienzo de cada sentencia, pero varias sesiones pueden tomar decisiones a partir de lecturas que luego quedan desactualizadas. SELECT ... FOR UPDATE espera ante un bloqueo incompatible y protege la fila hasta finalizar la transacción. El uso de una actualización condicionada y de controles de negocio sigue siendo necesario.

```
PS C:\Users\Lasag\OneDrive\Documentos\AaaTINO\Universidad\3° Año\Base de Datos\Proyecto Integrador\trabaj
o_integrador_bd> docker exec -it pg-bodega-paso7 psql -U postgres -d bodegas_atuel_paso7
psql (16.15 (Debian 16.15-1.pgdg13+2))
Type "help" for help.

bodegas_atuel_paso7=# SHOW transaction_isolation;
 transaction_isolation
-----------------------
 read committed
(1 row)

bodegas_atuel_paso7#
```

Confirmacion de aislamiento mediante SHOW transacción_isolation: read commited

7. Comparación y conclusión

Situación	Lo observado	Implicación
Simulación auxiliar sin bloqueo	Dos lecturas iguales; lógica insegura registra 14 salidas sobre 10	Riesgo de decidir con lectura antigua
Ultima botella sin FOR UPDATE	Ambas leen 1; A hace UPDATE 1; B espera y obtiene UPDATE 0	El UPDATE bloquea y reevalua la condición
Ultimas botellas con FOR UPDATE	A lee y bloquea 1; B espera; luego observa 0	La decisión se toma con stock actualizado
Procedimiento real de transferencia	Dos solicitudes de 7 sobre deposito de 10; solo una se proceso	La lógica transaccional rechaza faltante de stock

Las pruebas muestran que FOR UPDATE anticipa la coordinación desde la lectura; sin él, dos sesiones pueden leer el mismo stock, aunque los UPDATE siguen adquiriendo bloqueos. La simulación auxiliar muestra el problema de programar ventas a partir de lecturas antiguas sin una verificación segura. Los procedimientos del proyecto y las restricciones de stock, por su parte, evitan que las transferencias concurrentes produzcan saldos negativos o cambios parciales.

Alcance de la evidencia: el caso de última botella se reprodujo directamente sobre stock_sala, sin comprobantes. La prueba de transferencia corresponde al procedimiento real sp_procesar_transferencia. Todos los ensayos se realizaron en bases de pruebas separadas, sin alterar las migraciones del proyecto